

ASP.NET Core .NET 10 JWT 驗證與授權實戰

Authentication • Authorization • Claims • Role • Policy 完整實作

課程地圖

本課程依照真實 API 請求生命週期設計，從登入到授權回應，逐步建立完整的 JWT 驗證與授權實作能力。

01



登入

使用者發送帳密要求進行登入
驗證

02



身份驗證

後端系統驗證憑證並確認使用
者身份

03



Token 發行

驗證成功後產生並回傳 JWT
Token

04



身份主體

解析 Token 並建立
ClaimsPrincipal

05



授權

根據 Role 或 Policy 限制 API
存取

 每個環節皆對應課程中的實作章節，建議依序學習以確保理解完整流程。

學習目標

完成本課程後，
學員將具備以下核心能力：

01

理解 JWT

掌握 JWT 結構、運作原理與安全性設計

02

建立 JWT API

實作完整的 Login Endpoint 與 Token 發行邏輯

03

驗證 Bearer Token

設定 JwtBearer Middleware 進行 Token 驗證

04

使用 Claims

將 Claims 附加至 ClaimsPrincipal 並於 Controller 讀取

05

Role 與 Policy 授權

實作角色授權與自訂 Policy 授權，處理 403 情境

Authentication vs Authorization

兩者的根本差異: 先驗身份, 再判斷權限

Authentication (驗證)

Who are you?

- 確認使用者是否為合法身份
- 驗證帳號密碼、憑證或 Token
- 成功後產生 ClaimsPrincipal
- 失敗回應: **401 Unauthorized**

Authorization (授權)

What can you do?

- 確認已驗證的使用者擁有哪些權限
- 根據 Role、Claim 或 Policy 判斷
- 在 Authentication 完成後才執行
- 失敗回應: **403 Forbidden**

01

Client
發送請求

02

UseAuthentication
執行驗證

03

UseAuthorization
判斷權限

04

Controller
執行回應

Cookie、Session 與 JWT 比較

為什麼現代 API 選擇 JWT Stateless 架構？

Cookie

狀態
Stateful

儲存位置
瀏覽器 Cookie

擴展性
中

跨域支援
受限

適用場景
傳統 Web 應用

Session

狀態
Stateful

儲存位置
Server 記憶體

擴展性
低 (需共用 Store)

跨域支援
受限

適用場景
MVC 應用程式

JWT (Token)

狀態
Stateless

儲存位置
Client 端 (Header)

擴展性
高 (不需 Server 狀態)

跨域支援
原生支援

適用場景
Web API、微服務

i JWT 不在 Server 端儲存狀態，天然適合水平擴展的微服務架構與多端 (Web/Mobile/IoT) 情境。

JWT 結構解析

三段式設計: Header . Payload . Signature



Header

演算法類型與 Token 類型

```
{ "alg": "HS256",  
  "typ": "JWT" }
```



Payload

Claims 資料(使用者資訊、過期時間)

```
{ "sub": "user01",  
  "role": "Admin",  
  "exp": 1234567890 }
```



Signature

使用 SigningKey 對前兩段進行簽章

```
HMACSHA256(  
  base64(H)+"."+  
  base64(P), key)
```



Header 與 Payload 僅為 Base64Url 編碼, **不是加密**, 任何人皆可解碼讀取內容。請勿存放敏感資料如密碼。



安全性來源是 **Signature**。若資料被篡改, Server 端重新計算的簽章將不符, Token 驗證失敗。

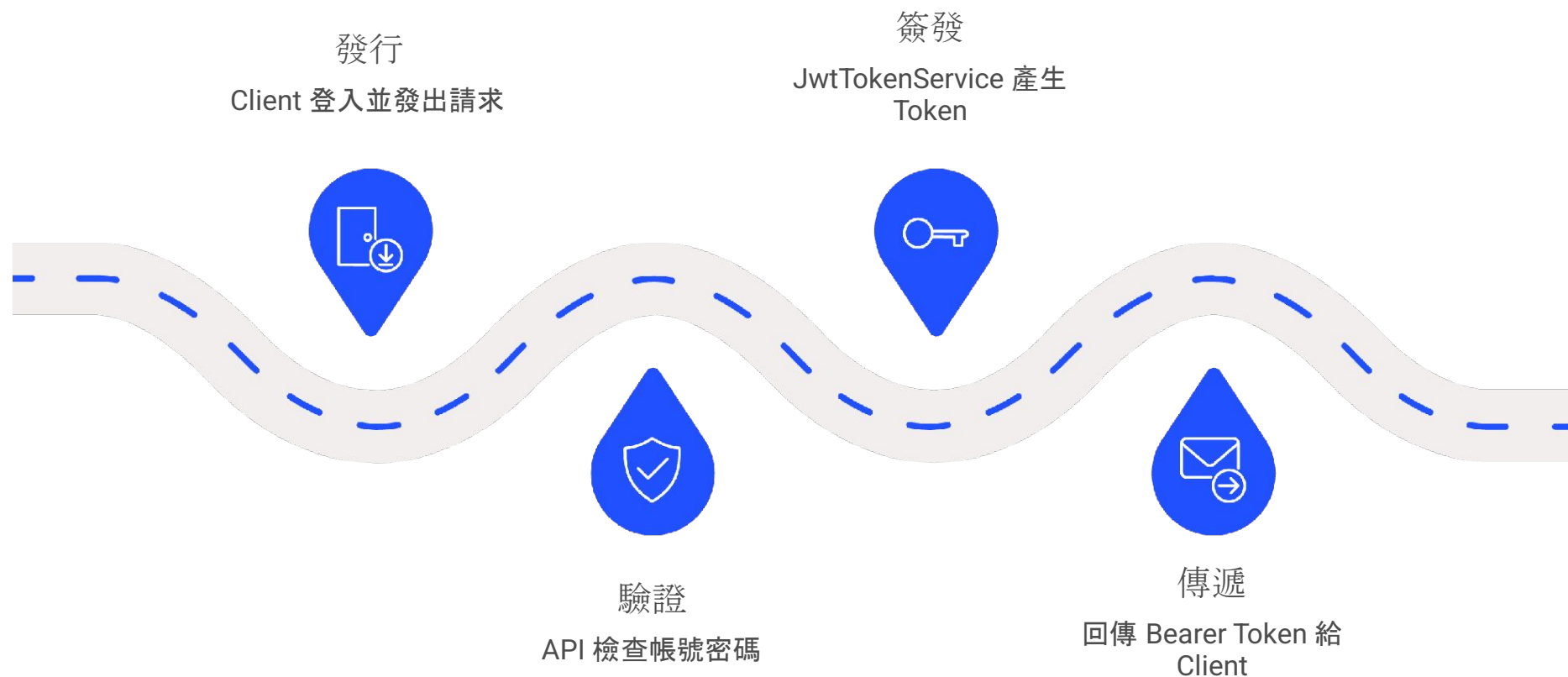
JWT Claims 參考表

標準 Registered Claims 與常見自訂 Claims

Claim	全名	用途說明
iss	Issuer	Token 的發行者，通常為 API 的 Base URL
aud	Audience	Token 的目標受眾，防止Token 被其他服務誤用
sub	Subject	Token 所代表的使用者唯一識別碼
name	Name	使用者顯示名稱
preferred_username	Preferred Username	登入帳號名稱 (OpenID Connect 標準)
role	Role	使用者角色，用於 Role-based Authorization
department	Department (自訂)	部門資訊，用於 Policy-based Authorization
exp	Expiration Time	Token 到期的 Unix 時間戳，超過則自動失效
iat	Issued At	Token 發行時間，可用於稽核追蹤
jti	JWT ID	Token 唯一識別碼，可用於防止Replay Attack

JWT 驗證完整流程

從 Client 登入到 ClaimsPrincipal 建立的完整生命週期



步驟 1-3: 發行

帳密驗證通過後, 由 Token Service 簽發 JWT

步驟 4-6: 傳輸與驗證

Client 在每個請求的 Header 附上 Bearer Token

步驟 7-8: 授權

Middleware 解析 Claims, 框架自動完成授權判斷

專案架構設計

責任分離原則: 每個資料夾負責單一職責

Controllers/

處理 HTTP 請求路由, 呼叫 Service, 回傳 HTTP 回應。不包含業務邏輯。

Services/

核心業務邏輯: DemoUserService (使用者驗證)、JwtTokenService (Token 產生)。

DTOs/

定義 API 輸入輸出資料結構, 避免直接暴露 Domain Model。

Program.cs

應用程式進入點

負責全域配置與初始化:

- DI 容器設定
- Middleware Pipeline 組裝
- Authentication 驗證配置
- Authorization 授權初始化

Filters/

Action Filter 或 Exception Filter, 可用於統一錯誤處理與記錄。

Options/

強型別設定類別, 對應 appsettings.json 的 JWT 區段。

Models/

資料庫實體或 Domain 模型, 與 DTO 分開維護, 避免耦合。

安全管理 SigningKey: User Secrets

絕對不要將 **SigningKey** 存放在 **appsettings.json**

❗問題: appsettings.json 的風險

- 設定檔通常會被提交至Git Repository
- 任何有 Repo 存取權的人都能取得 **SigningKey**
- **SigningKey 洩漏** = 任何人都能偽造有效Token
- 一旦外洩, 所有現有Token 皆無法信任

解決方案: 依環境分層管理

01開發環境: User Secrets

使用 `dotnet user-secrets set` 指令, 秘密儲存於本機, 不進版控。

02測試 / CI 環境: 環境變數

透過 CI/CD 管道注入, 不寫入任何設定檔。

03正式環境: Azure Key Vault

整合 Managed Identity, 零憑證存取, 集中稽核管理。



永遠不要在 **appsettings.json**、**appsettings.Production.json** 中存放任何密鑰或連線字串密碼。

JWT Options 設定類別

強型別設定: 使用 **Options Pattern** 封裝 JWT 參數

```
// Options/JwtOptions.cs
public class JwtOptions
{
    public string Issuer { get; set; } = string.Empty;
    public string Audience { get; set; } = string.Empty;
    public string SigningKey { get; set; } = string.Empty;
    public int ExpirationMinutes { get; set; } = 60;
}
```

```
// appsettings.json (不含 SigningKey)
{
  "Jwt": {
    "Issuer": "https://api.example.com",
    "Audience": "https://app.example.com",
    "ExpirationMinutes": 60
  }
}
```

Issuer

Token 發行者識別, 通常為 API 的根 URL。驗證時必須完全比對。

Audience

Token 目標受眾識別。防止 Token 被其他服務接受使用。

SigningKey

HMAC 對稱簽章密鑰, 長度建議 ≥ 256 bits (32 bytes)。僅存於 User Secrets / Key Vault。

ExpirationMinutes

Token 有效期限(分鐘), 建議短期 (15–60 分鐘), 搭配 Refresh Token 使用。

Program.cs 設定(一): 加入 Authentication

使用 `AddAuthentication` 與 `AddJwtBearer` 完成服務註冊

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// 1. 繫結 JWT Options
builder.Services.Configure<JwtOptions>(
    builder.Configuration.GetSection("Jwt"));

// 2. 註冊 Token Service
builder.Services.AddScoped<IJwtTokenService,
    JwtTokenService>();
builder.Services.AddScoped<IDemoUserService,
    DemoUserService>();

// 3. 加入 Authentication
builder.Services
    .AddAuthentication(JwtBearerDefaults
        .AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        // → 見下一頁 TokenValidationParameters
    });
```

Configure<JwtOptions>

將 appsettings.json 的 "Jwt" 區段繫結至強型別 `JwtOptions`, 透過 DI 注入使用。

AddScoped Services

將 Token Service 與 User Service 以 Scoped 生命週期註冊, 每個 HTTP Request 獨立實例。

AddAuthentication

指定預設 Authentication Scheme 為 `JwtBearer`, 框架處理未驗證請求時以此 Scheme 回應 401。

Program.cs 設定 (二) : TokenValidationParameters

每個驗證參數都有明確的安全意義

```
.AddJwtBearer(options =>
{
    var jwtOpts = builder.Configuration
        .GetSection("Jwt")
        .Get<JwtOptions>()!;

    options.TokenValidationParameters =
        new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = jwtOpts.Issuer,

            ValidateAudience = true,
            ValidAudience = jwtOpts.Audience,

            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero,

            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwtOpts.SigningKey)),

            NameClaimType = "name",
            RoleClaimType = "role"
        };
});
```

ValidateIssuer / Audience

防止來自其他服務的 Token 被本 API 接受。

ValidateLifetime

檢查 exp Claim, 過期 Token 自動拒絕。

ClockSkew = Zero

預設有 5 分鐘寬容, 設為 0 確保精確過期行為。

ValidateIssuerSigningKey

驗證 Signature, 防止偽造或篡改的 Token 通過。

NameClaimType / RoleClaimType

對應自訂 Claim 名稱至 User.Identity.Name 與 User.IsInRole()。

Token 驗證參數詳解

理解每個驗證開關的用途與關閉的風險

參數	預設值	用途	關閉風險
ValidateIssuer	true	確認 Token 的 iss Claim 符合設定值	其他服務的 Token 可被接受
ValidateAudience	true	確認 Token 的 aud Claim 符合設定值	Token 可被跨服務重複使用
ValidateLifetime	true	檢查 exp 是否已超過當前時間	過期 Token 永久有效
ValidateIssuerSigningKey	false	驗證 Signature 是否由合法 Key 簽發	任何偽造 Token 均可通過
ClockSkew	5 分鐘	允許 Server 時間輕微誤差	設為 0 確保精確過期

⊗ 生產環境中, ValidateIssuerSigningKey 必須設為 true, 否則 Signature 驗證形同虛設。

Middleware Pipeline 順序

Use 的順序決定了請求的處理流程，順序錯誤將導致授權失效

✓ 正確順序

```
app.UseAuthentication();  
app.UseAuthorization();  
app.MapControllers();
```

Authentication 先執行，解析 Token 並建立 **ClaimsPrincipal**，Authorization 才能判斷角色與 Policy。

✗ 常見錯誤：順序顛倒

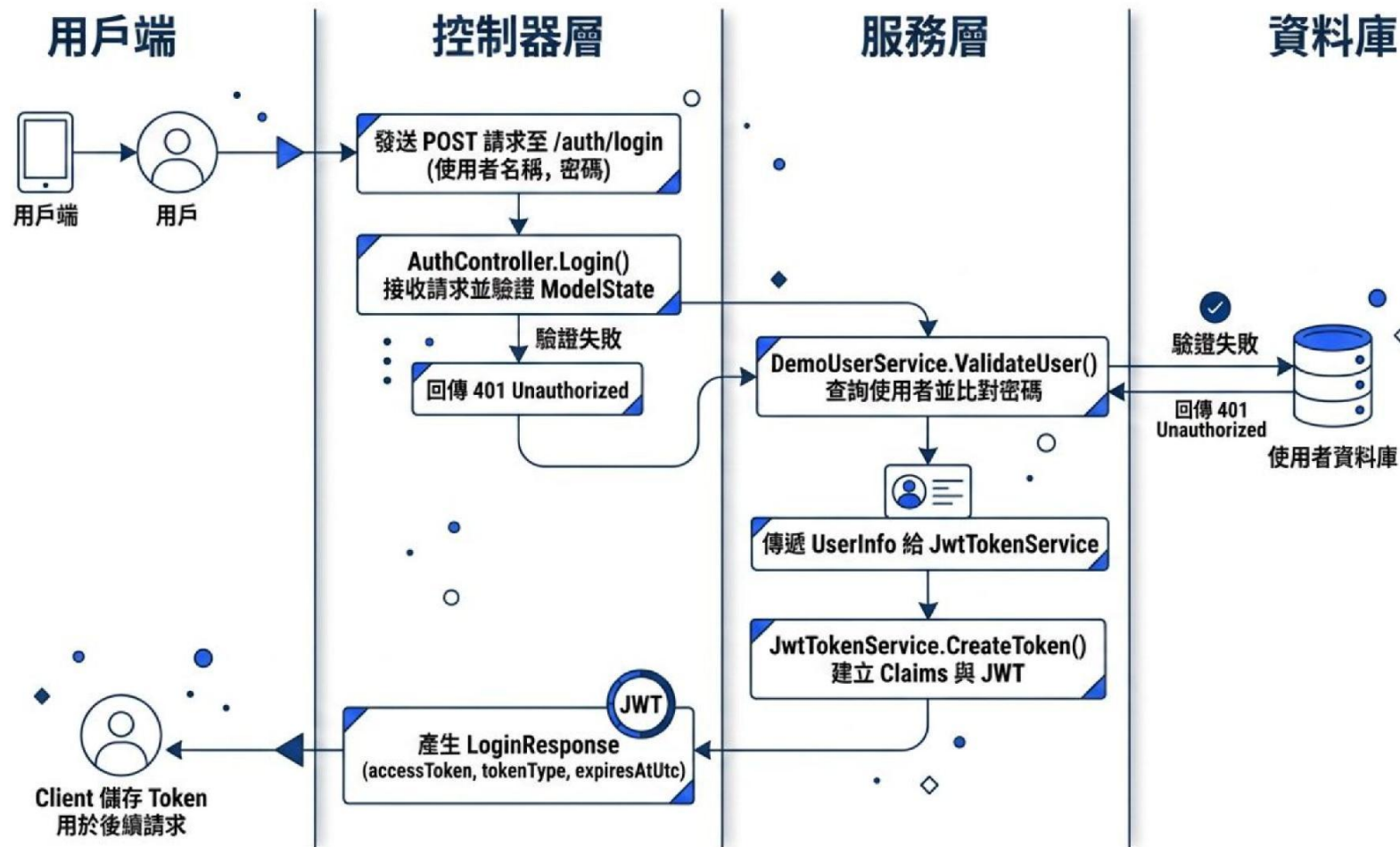
```
app.UseAuthorization(); // 錯誤  
app.UseAuthentication(); // 太晚了
```

Authorization 執行時 ClaimsPrincipal 尚未建立，所有帶有 **[Authorize]** 的 Endpoint 皆回傳 401，即使 Token 有效也無法通過。

⚠ 這是初學者最常見的設定錯誤，症狀為 Token 正確但始終 401。

登入流程 Sequence Diagram

AuthController → DemoUserService → JwtTokenService 完整呼叫鏈



JwtTokenService 實作

CreateToken() 核心邏輯: Claims → Credentials → Token

```
JwtTokenService.cs

// Services/JwtTokenService.cs
public string CreateToken(UserInfo user)
{
    var claims = new[]
    {
        new Claim("sub", user.Id),
        new Claim("name", user.DisplayName),
        new Claim("preferred_username", user.UserName),
        new Claim("role", user.Role),
        new Claim("department", user.Department),
        new Claim("jti", Guid.NewGuid().ToString())
    };

    var key = new
    SymmetricSecurityKey(Encoding.UTF8.GetBytes(_opts.SigningKey));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
        issuer: _opts.Issuer,
        audience: _opts.Audience,
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(_opts.ExpirationMinutes),
        signingCredentials: creds
    );
    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

01

建立 Claims 陣列

將使用者資訊轉換為Claim, 包含識別資訊、角色、部門與唯一JTI。

02

建立 SigningCredentials

使用 `SymmetricSecurityKey` 搭配 HMAC SHA-256 演算法建立簽章憑證。

03

建立 JwtSecurityToken

組裝 Token 物件, 設定 Issuer、Audience、Claims 與過期時間。

04

WriteToken 序列化

使用 `JwtSecurityTokenHandler` 將 Token 物件序列化為標準三段字串。

Login Response 格式

回傳標準化的JSON 結構, 方便 Client 端解析與儲存

```
// DTO/LoginResponse.cs
public record LoginResponse(
    string AccessToken,
    string TokenType,
    DateTime ExpiresAtUtc
);

// HTTP 200 OK Response
{
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
    "tokenType": "Bearer",
    "expiresAtUtc": "2025-10-01T10:15:00Z"
}
```

```
// 後續請求 Header
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

accessToken

JWT 字串, 每次API 請求放入 Authorization Header 的 Bearer 值中。

tokenType

固定值 "Bearer", 符合 OAuth 2.0 Bearer Token 標準, 告知 Client 如何使用。

expiresAtUtc

Token 過期時間(UTC), Client 可提前判斷是否需要Refresh, 避免以過期 Token 呼叫 API。



建議 Client 在 expiresAtUtc 前 1-2 分鐘主動更新Token, 避免因 ClockSkew 造成邊界錯誤。

[Authorize] 屬性的行為

未加 [Authorize] 為公開端點;加上後框架自動進行身份驗證檢查

```
// 無需驗證的公開端點
[AllowAnonymous]
[HttpPost("login")]
public IActionResult Login(...)

// 需要驗證的受保護端點
[Authorize]
[HttpGet("profile")]
public IActionResult GetProfile()
{
    var name = User.FindFirstValue("name");
    return Ok(new { Name = name });
}

// 全域套用(Controller 層級)
[Authorize]
[ApiController]
[Route("api/[controller]")]
public class SecureController : ControllerBase
```



ClaimsPrincipal 讀取使用者資訊

在 **Controller** 中透過 **HttpContext.User** 存取 JWT Claims

```
// 在 Controller 中讀取 Claims
[Authorize]
[HttpGet("me")]
public IActionResult GetMe()
{
    // 方法一:FindFirstValue(最常用)
    var name = User.FindFirstValue("name");
    var username = User
        .FindFirstValue("preferred_username");
    var role = User.FindFirstValue("role");
    var dept = User.FindFirstValue("department");

    // 方法二:Identity.Name(對應 NameClaimType)
    var displayName = User.Identity?.Name;

    // 方法三:IsInRole(對應 RoleClaimType)
    var isAdmin = User.IsInRole("Admin");

    // 方法四:所有 Claims
    var allClaims = User.Claims
        .Select(c => new { c.Type, c.Value });

    return Ok(new { name, username, role, dept });
}
```

HttpContext.User

型別為 ClaimsPrincipal, 由 JwtBearer Middleware 解析 Token 後自動填充, 在整個請求生命週期中可用。

FindFirstValue()

依 Claim Type 名稱取得第一個符合的值, 回傳 string?, 是最常用的讀取方式。

Identity.Name

取得 NameClaimType 設定對應的 Claim 值, 預設為 ClaimTypes.Name, 可自訂。

Role-Based Authorization

最簡單的授權方式: 依角色名稱限制存取

```
// 單一角色限制
[Authorize(Roles = "Admin")]
[HttpGet("admin/users")]
public IActionResult GetAllUsers() { ... }

// 多角色 (OR 邏輯, 任一符合即通過)
[Authorize(Roles = "Admin,Manager")]
[HttpGet("reports")]
public IActionResult GetReports() { ... }

// 多重限制 (AND 邏輯, 全部必須符合)
[Authorize(Roles = "Admin")]
[Authorize(Roles = "SuperUser")]
[HttpDelete("admin/user/{id}")]
public IActionResult DeleteUser(string id)
{ ... }
```

Admin 角色

最高權限, 可存取所有管理功能與使用者資料。

Student 角色

一般使用者, 僅能存取自己的資料與公開課程資源。

 多角色以逗號分隔為 **OR** 邏輯; 疊加多個 `[Authorize]` 屬性為 **AND** 邏輯。

 401 = 未登入 / Token 無效; 403 = 已登入但角色不符。兩者意義截然不同。

Policy-Based Authorization

比 **Role** 更彈性: 依自訂邏輯定義細粒度存取規則

```
// Program.cs: 定義 Policy
builder.Services.AddAuthorization(options =>
{
    // 依 Claim 值判斷
    options.AddPolicy("EngineeringOnly", policy =>
        policy.RequireClaim("department",
            "Engineering"));

    // 組合多個條件 (AND 邏輯)
    options.AddPolicy("SeniorEngineer", policy =>
        policy
            .RequireRole("Engineer")
            .RequireClaim("department",
                "Engineering")
            .RequireClaim("level", "Senior"));

    // 自訂需求 (進階)
    options.AddPolicy("MinAge18", policy =>
        policy.Requirements.Add(
            new MinimumAgeRequirement(18)));
});

// Controller: 套用 Policy
[Authorize(Policy = "EngineeringOnly")]
[HttpGet("engineering/specs")]
public IActionResult GetSpecs() { ... }
```

01

定義 **Policy** (**Program.cs**)

在 `AddAuthorization` 中以 `AddPolicy` 命名並定義條件, 支援 `Role`、`Claim` 與自訂 `Requirement`。

02

套用 **Policy** (**Controller**)

在 `[Authorize]` 屬性中指定 `Policy` 參數名稱, 與定義時的 `Key` 對應。

03

框架自動評估

請求到達時, `Authorization Middleware` 自動評估所有條件, 不符合則回傳 403。

Role vs Policy 比較

選擇適合情境的授權策略, 兩者可混合使用

比較項目	Role Authorization	Claim Authorization	Policy Authorization
判斷依據	使用者角色名稱	特定 Claim 存在與值	複合條件 (Role + Claim + 自訂)
設定位置	Attribute 直接指定	Attribute 或 Policy	Program.cs 集中定義
彈性程度	低 (固定角色名稱)	中 (依 Claim 值)	高 (可組合任意條件)
維護性	簡單, 但角色多時難管理	中	集中管理, 易於修改
適用情境	簡單系統、2-5 個角色	依屬性判斷存取	企業系統、細粒度權限
使用範例	<code>Roles="Admin"</code>	<code>RequireClaim("dept")</code>	<code>Policy="SeniorEngineer"</code>

建議: 小型系統以 Role 快速實作; 企業系統採 Policy 統一管理, 便於後期維護與擴展。

Swagger 整合 JWT Demo

在 Swagger UI 中配置 Bearer Token 進行 API 測試

```
// Program.cs:設定 Swagger 支援 JWT
builder.Services.AddSwaggerGen(options
=>
{
    options.AddSecurityDefinition("Bearer",
        new OpenApiSecurityScheme
        {
            Name = "Authorization",
            Type = SecuritySchemeType.Http,
            Scheme = "bearer",
            BearerFormat = "JWT",
            In = ParameterLocation.Header,
            Description = "輸入 JWT Token"
        });
    options.AddSecurityRequirement(
        new OpenApiSecurityRequirement {
            ... });
});
```

01

POST /auth/login

在 Swagger 中輸入帳號密碼, 取得 accessToken。

02

點擊 Authorize 按鈕

在 Swagger UI 右上角輸入 Token(不需加 "Bearer" 前綴, 設定已自動處理)。

03

呼叫受保護 API

Swagger 自動在 Header 附加 Authorization: Bearer ..., 可直接測試 200/401/403 回應。



HTTP 請求測試工具

使用 **.http** 檔案與 **curl** 進行 API 測試

.http 檔案 (VS Code REST Client)

```
### 1. 登入取得 Token
POST https://localhost:5001/auth/login
Content-Type: application/json

{
  "username": "admin",
  "password": "P@ssw0rd"
}

### 2. 呼叫受保護端點
GET https://localhost:5001/api/profile
Authorization: Bearer {{accessToken}}

### 3. 呼叫 Admin 專屬端點
GET https://localhost:5001/api/admin/users
Authorization: Bearer {{accessToken}}
```

curl 指令

```
# 登入
curl -X POST https://localhost:5001/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"P@ssw0rd"}'

# 使用 Token 呼叫 API
TOKEN="eyJhbGciOiJIUzI1NiIs..."

curl https://localhost:5001/api/profile \
  -H "Authorization: Bearer $TOKEN"

# 測試 Admin Endpoint
curl https://localhost:5001/api/admin/users \
  -H "Authorization: Bearer $TOKEN" \
  -v # -v 顯示完整回應 Header
```

 推薦使用 VS Code 的 **REST Client 擴充套件**，可將測試請求與專案一同版控，方便團隊共用。

401 vs 403 完整比較

正確理解兩種 HTTP 狀態碼, 避免錯誤的錯誤訊息回傳

401 Unauthorized

「你是誰？我不認識你。」

- 未提供 Authorization Header
- Token 格式不正確
- Token 已過期(exp 超過)
- Signature 驗證失敗(被篡改)
- Issuer 或 Audience 不符

對應 ASP.NET Core 回應: JwtBearer Middleware 自動回傳 401, 附帶 WWW-Authenticate: Bearer Header。

403 Forbidden

「我認識你, 但你沒有權限。」

- Token 有效, 身份已確認
- 但角色不符合 [Authorize(Roles)]
- 或 Policy 條件不滿足
- 明確知道使用者是誰, 但拒絕存取

對應 ASP.NET Core 回應: Authorization Middleware 評估失敗後回傳 403, 不附帶驗證挑戰 Header。

情境	401	403
建議 Client 動作	重新登入, 取得新Token	提示使用者權限不足, 聯繫管理員

JWT Debug Checklist

遇到 401 時, 依序檢查這 8 個常見問題點

1 Authorization Header 格式

確認請求 Header 包含 Authorization, 格式為 Bearer <token> (注意大小寫與空格)。

2 Bearer 前綴

Bearer 後需有一個空格, Token 直接跟在後面, 中間無多餘字元。

3 Issuer / Audience 比對

Token 中的 iss 與 aud 必須與 TokenValidationParameters 設定的值完全一致 (含大小寫與尾部斜線)。

4 Signature 與 SigningKey

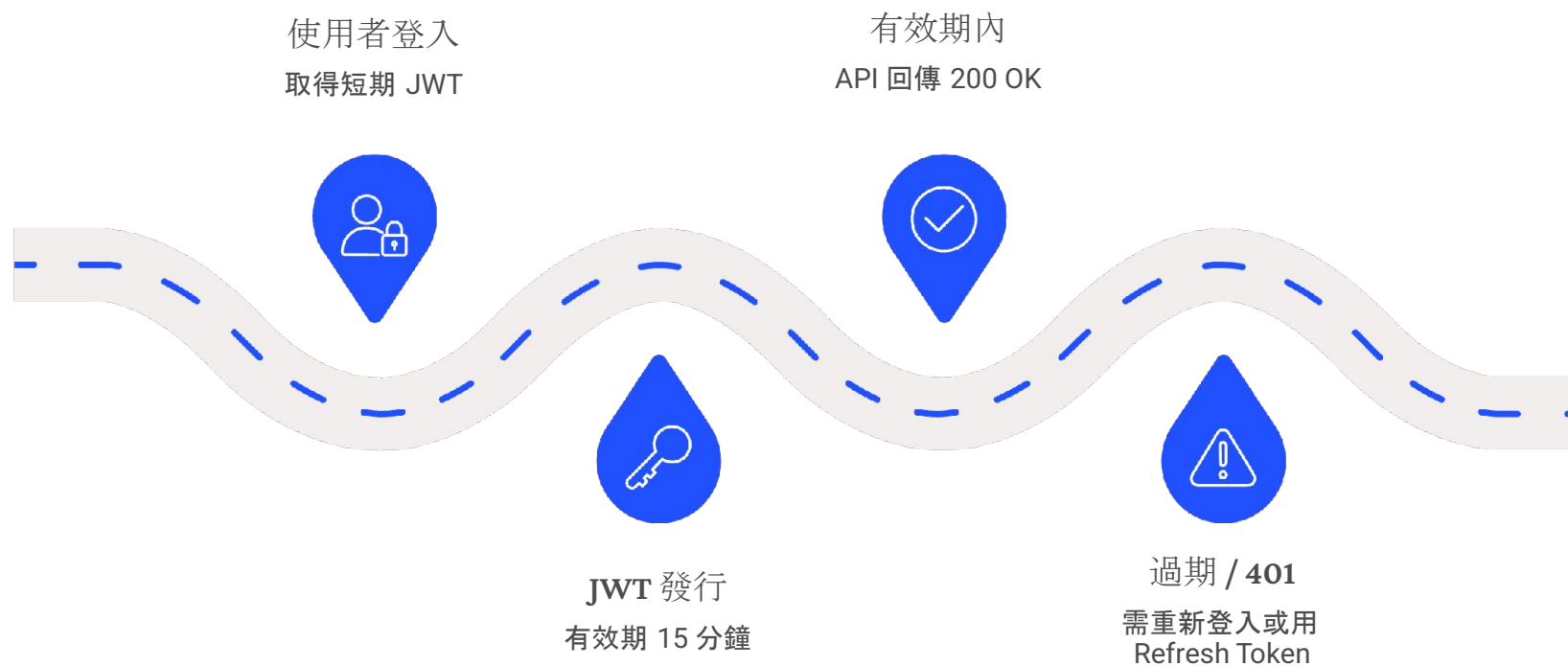
簽發 Token 時使用的 Key 與驗證時的 IssuerSigningKey 必須完全相同, 包含編碼方式 (UTF-8)。

5 Token 過期與 ClockSkew

確認 Server 時間正確 (建議使用 NTP), 並檢查 ClockSkew 設定。可先調高過期時間確認問題是否在此。

Token 過期示意

Token 有有效期限, 過期後必須重新取得



短期 Token (推薦)

15-60 分鐘過期。降低 Token 洩漏後的攻擊窗口, 搭配 Refresh Token 使用。

長期 Token (不推薦)

24 小時以上。實作簡單但風險高, 一旦 Token 洩漏, 攻擊者有更長的利用時間。



JWT 本身無法撤銷 (Revoke)。在 Token 過期前若需要強制失效, 需搭配 Token 黑名單 (如 Redis) 或改用 Refresh Token 機制。

實作 Lab: 完整練習流程

按照 **Checklist** 依序完成所有驗證情境

1

登入取得 **Token**

POST `/auth/login` 以正確帳密取得 `accessToken`, 驗證回傳格式正確。

2

讀取個人資料

GET `/api/profile` 攜帶 Token, 確認能讀取 `name`、`role`、`department` Claims。

3

Admin 角色驗證

以 Admin 帳號呼叫 `[Authorize(Roles="Admin")]` 端點, 確認 200; 以 Student 帳號呼叫, 確認 403。

4

Policy 授權驗證

以工程部門帳號呼叫 `EngineeringOnly` Policy 端點, 確認通過; 以其他部門帳號測試 403。

5

修改 **Token** 測試

手動修改 JWT Payload (如更改 role), 確認 Server 回傳 401 (Signature 驗證失敗)。

Challenge: 進階實作挑戰

完成基礎 Lab 後, 嘗試以下進階挑戰

新增 **Teacher** 角色

在 DemoUserService 新增 Teacher 測試帳號, 設定 role = "Teacher", 建立 TeacherOnly Policy, 限制只有 Teacher 角色才能存取課程管理 API。

Department Policy

新增 department = "Education" Claim, 定義 EducationDept Policy, 並建立一個需要同時具備 Teacher 角色 AND Education 部門的複合 Policy。

01

新增 **Teacher** 帳號

在 DemoUserService 加入測試使用者

02

更新 **Token Service**

確認 department Claim 正確寫入 JWT

03

定義 **Policy**

在 Program.cs 的 AddAuthorization 中加入新 Policy

04

建立受保護端點

新增 TeacherController 並套用 Policy

05

Swagger 測試

以不同角色帳號測試, 確認 200 / 403 行為正確

課堂測驗(1-5 題)

選出最正確的答案

Q1

JWT 的 Payload 是？

- A. 使用 AES 加密的密文 B. 使用 Base64Url 編碼的 JSON ☒ C. Base64Url 編碼, 可被任何人解碼 D. 只有 Server 能讀取

Q2

JWT 的安全性主要來自？

- A. Base64 加密 B. HTTPS 傳輸 ☒ C. Signature 的 SigningKey D. Token 的長度

Q3

Middleware 正確順序為？

- A. Authorization → Authentication ☒ B. Authentication → Authorization → MapControllers C. MapControllers → Authentication D. 順序無影響

Q4

HTTP 403 代表？

- A. 未提供 Token B. Token 已過期 C. Signature 錯誤 ☒ D. Token 有效但無此操作的權限

Q5

ClockSkew 的作用是？

- A. 加密時間戳 ☒ B. 允許 Server 與 Client 時間輕微誤差 C. 延長 Token 有效期 D. 設定 Token 簽發時間

課堂測驗 (6–10 題)

選出最正確的答案

Q6: Role 與 Policy 的主要差異？

- A. Role 比 Policy 安全 ☒ B. Policy 可組合多種條件, 彈性更高 C. Policy 只能用於 Admin D. Role 需在 Program.cs 定義

Q7: SigningKey 應儲存在哪裡？

- A. appsettings.json B. 程式碼常數 ☒ C. User Secrets (開發) / Azure Key Vault (正式) D. 資料庫

Q8: jti Claim 的用途是？

- A. 記錄登入時間 B. 儲存使用者部門 ☒ C. Token 唯一識別碼, 可防止 Replay Attack D. 定義 Token 類型

Q9: NameClaimType 設定的作用是？

- A. 設定 Token 名稱 ☒ B. 對應

正式環境安全考量

從課堂 **Demo** 到生產環境的關鍵差距



OIDC / OAuth 2.0

正式環境不自行實作 Authentication, 改用標準協議整合 Identity Provider (Azure AD、Keycloak、Auth0), 降低自行實作的安全風險。



Azure Key Vault

所有密鑰、憑證、連線字串集中於 Key Vault 管理, 搭配 Managed Identity 零憑證存取, 提供完整稽核記錄。



HTTPS + Rate Limiting

所有流量必須使用 TLS 1.2+, 並對 Login Endpoint 設定 Rate Limit, 防止暴力破解攻擊。



Audit Log

記錄所有登入成功、失敗、Token 使用與授權拒絕事件, 符合企業合規與資安稽核需求。

Refresh Token 機制

解決短期 **Access Token** 過期問題，無需重新登入

Refresh Token 運作流程

01

使用 API

Access Token 有效時，API 正常回傳 200

02

Token 過期

Access Token 到期後，API 拒絕存取並回傳 401

03

換發 Token

Client 呼叫 `/auth/refresh` 提交檢驗 Refresh Token

04

Token Rotation

驗證成功後，發行新 Access 與 Refresh，並廢棄舊 Token

核心安全機制

Token Rotation (輪替機制)

每次使用 Refresh Token 時，同時發行新的 Refresh Token 並使舊的失效，降低長期憑證被竊取的風險。

Revoke (主動撤銷)

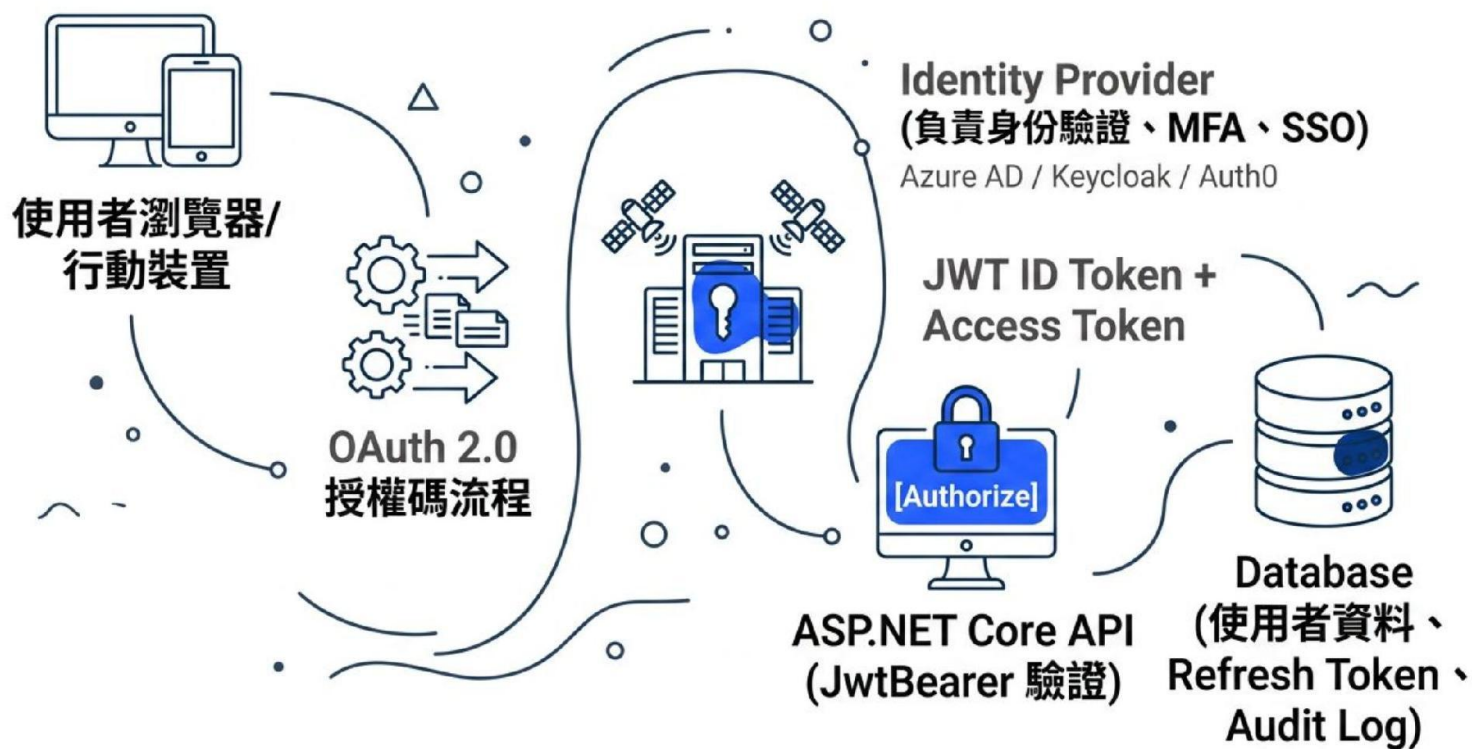
Refresh Token 儲存於 Server 端資料庫中，管理者可主動撤銷其效力，完美解決傳統JWT無法即時撤銷的問題。

Replay Attack 防護 (重放攻擊防禦)

若攻擊者截取並試圖重放舊的 Refresh Token，Rotation 偵測機制會立即識別出異常，並自動撤銷該家族發行的所有Token。

正式登入架構

整合 OIDC Identity Provider 的企業級完整架構



Identity Provider 的責任

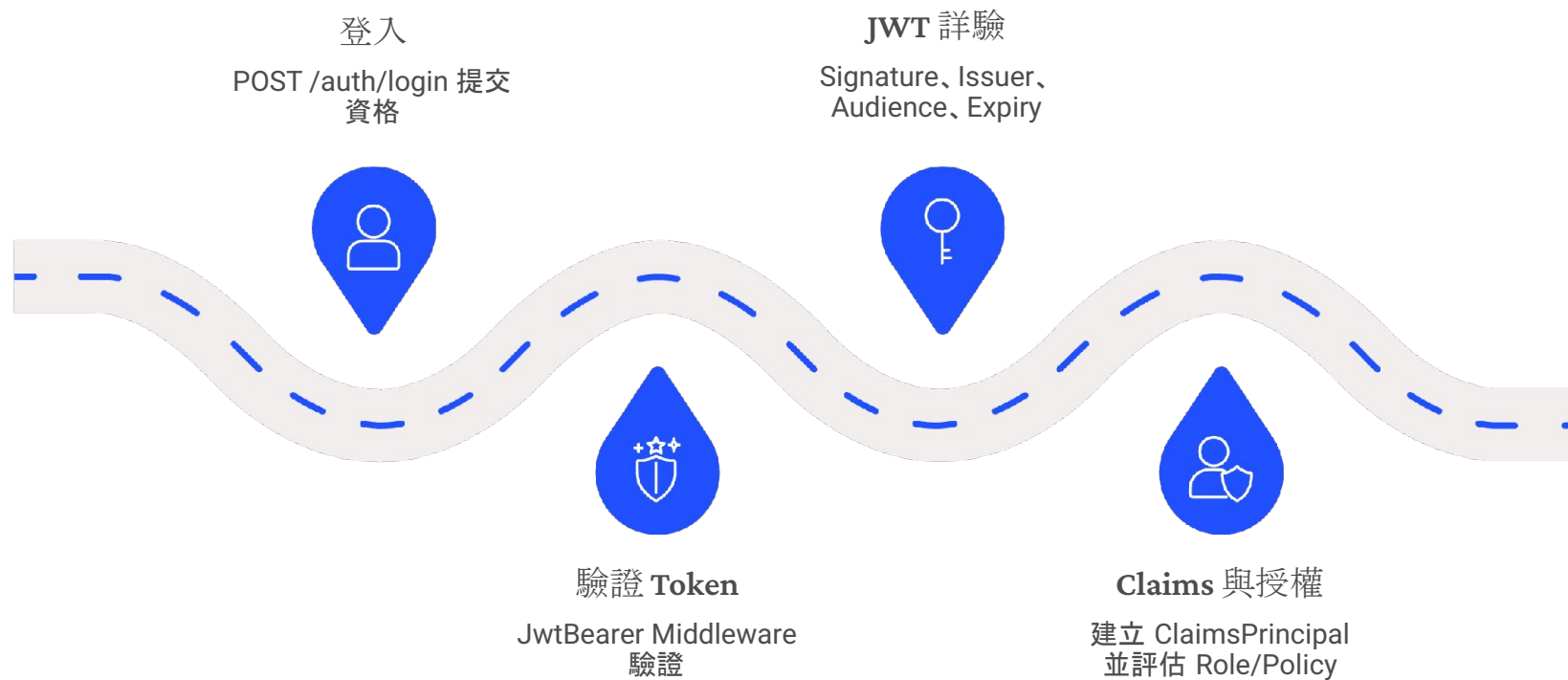
MFA、SSO、帳號管理、密碼政策、登入事件稽核。API 不處理帳密，降低攻擊面。

API 的責任

驗證 JWT 有效性、解析 Claims、執行 Authorization、回傳業務資料。不管理身份。

課程總結

回顧完整的 JWT 驗證與授權流程



🔑 JWT 理論

Header.Payload.Signature 三段結構, Base64Url 非加密, Signature 是安全核心

🛡️ 授權策略

Role 快速實作, Policy 企業彈性, 401 vs 403 精確區分

⚙️ 實作設定

Options Pattern、TokenValidationParameters、Middleware 順序

🚀 正式環境

OIDC、Key Vault、Refresh Token、Rate Limit、Audit Log

學習資源與延伸閱讀

繼續深化 **ASP.NET Core** 安全性知識的推薦路徑

Microsoft Learn 官方文件

learn.microsoft.com/aspnet-core/security – ASP.NET Core 安全性完整文件，包含 Authentication、Authorization、Data Protection 等章節。

JWT 規範

RFC 7519 (JWT)、RFC 7517 (JWK)、RFC 6749 (OAuth 2.0) – 了解標準背後的设计原則與安全考量。

jwt.io

線上 JWT 解碼與驗證工具，可即時視覺化 Token 的三個組成部分，適合偵錯使用。

延伸閱讀主題

- ASP.NET Core Identity – 完整的使用者管理框架
- Azure AD B2C – 企業消費者身份管理
- OpenIddict – 自建 OpenID Connect Server
- IdentityServer / Duende – 企業級授權伺服器

下一步建議

- 完成所有 Lab 練習並通過 Challenge
- 在現有專案中導入 JWT 取代 Session
- 研究 OIDC 授權碼流程
- 評估導入 Azure AD 整合

完成本課程...

ASP.NET Core .NET 10 JWT 驗證與授權實戰

今日重點回顧

- 01 JWT 三段結構與安全原理
- 02 完整 Authentication Pipeline 設定
- 03 Role 與 Policy 授權實作
- 04 401 / 403 的正確處理

帶走的能力

- 01 可獨立實作 JWT API 專案
- 02 能設計細粒度授權策略
- 03 具備正式環境安全意識
- 04 可作為團隊 JWT 知識傳承者

Authentication • Authorization • Claims • Role • Policy

在實際專案中持續實踐，是鞏固今日學習的最佳途徑。